

Word-level Plagiarism Checker for Short documents

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
M.Ananya (2520080069)
P.Ruhitha (2520080053)

Team Number: 9

Section: 10

Under the guidance of

Dr. Chandusha Kanda

Department of Computer Science and Information Technology



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University
DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled “**Word-level Plagiarism Checker for Short documents**” is a record of Bonafide work of **M.Ananya – 2520080069**, **P.Ruhitha – 2520080053** submitted in partial fulfillment for the award of B. Tech in Artificial Intelligence & Data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

M.Ananya – 2520080069

P.Ruhitha – 2520080053

K L (Deemed to be) University

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE



CERTIFICATE

This is to certify that the mini project based report entitled “**Word-level Plagiarism Checker for Short documents**” is a bonafide work done and submitted by **M.Ananya – 2520080069**, **P.Ruhitha – 2520080053** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Supervisor

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, _Dr. Chandusha Kanda_, Assistant Professor, Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Sreenath Itikela (AI&DS)**, Head of the Department of Artificial Intelligence and Data Science for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

M.Ananya – 2520080069

P.Ruhitha – 2520080053

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	18
8.	Conclusion & Future Scope	21
9.	References	22

1.ABSTRACT

Plagiarism checking of academic documents depends on how quickly and accurately the text of a submitted document can be compared with existing literature. This project, Word-level Plagiarism Checker for Short documents, is a Data Structures and Algorithms based system that compares an uploaded PDF document with reference papers fetched automatically from Semantic Scholar. The text of the uploaded PDF is extracted using Apache PDFBox, converted into lowercase word tokens and split into overlapping word-level n-grams (trigrams, $n = 3$). A search query is built from the most frequent significant words of the document and is used to find open-access reference papers (up to three). The text of each reference paper is indexed in a custom hash table with separate chaining, and every trigram of the uploaded document is looked up in it to mark the matched words. The similarity score is the percentage of matched words out of the total words of the uploaded document. In addition, the Knuth-Morris-Pratt (KMP) and Rabin-Karp string matching algorithms are run on the same trigrams and compared on matches found, comparisons made and execution time. The backend is written in Java and the results are shown on a React-based dashboard named PlagiaSense, which displays the plagiarism score, matched text, DSA algorithm comparison and reference papers.

Team Name: 9

Team Members:

1. M.Ananya, 2520080069
2. P.Ruhitha, 2520080053

2. INTRODUCTION

Plagiarism is the use of another author's written work without proper acknowledgement. In academic writing, a document is therefore usually compared against existing research papers before it is accepted. Doing this comparison manually is slow, because the reviewer has to find suitable papers and read them side by side with the document. This project, Word-level Plagiarism Checker for Short documents, automates this comparison for PDF documents: the user uploads a PDF, and the system finds reference papers by itself, compares the text word by word and reports how much of the uploaded document is matched.

The core of the project is built on Data Structures and Algorithms. The text of a PDF is extracted and tokenised into lowercase words, and the words are grouped into overlapping sequences of three consecutive words (word-level trigrams). The trigrams of a reference paper are stored in a hash table implemented from scratch with separate chaining; each node keeps the trigram and the list of positions where it occurs. The trigrams of the uploaded document are then searched in this hash table. Every word that belongs to at least one matching trigram is counted as a matched word, and the similarity percentage is the number of matched words divided by the total number of words.

Reference papers are not supplied by the user. A search query is built from the most frequent significant words of the uploaded document and sent to the Semantic Scholar API [9]; the open-access PDFs of the papers found are downloaded, their text is extracted, and the document is analysed against each of them (up to three). The paper with the highest similarity gives the overall score. As a Data Structures and Algorithms comparison, the Knuth-Morris-Pratt (KMP) and Rabin-Karp string matching algorithms are also run on the same trigrams, and their matches, comparisons and execution time are compared.

The system has two parts: a Java backend (an HTTP server on port 8080 that uses Apache PDFBox [10] for PDF text extraction and Gson [11] for JSON) and a React dashboard named PlagiaSense built with React and Vite [12], which uploads the PDF and displays the results.

3. LITERATURE SURVEY

1. MOSS (Measure Of Software Similarity), Stanford University. MOSS is an automatic system for determining the similarity of programs. Its main application has been detecting plagiarism in programming classes, and it is provided as an Internet service where the user supplies a list of files to compare. MOSS is based on the winnowing document fingerprinting algorithm [2]. Unlike our project, which compares the natural-language words of PDF documents, MOSS works on program source code [1].

2. JPlag. JPlag is a system that finds similarities among sets of source code files and can detect software plagiarism and collusion. It compares programs structurally rather than as plain text, by converting the code into token sequences, so renaming variables does not hide copying. It is mainly used in programming education. Our project instead compares word sequences of a document with papers fetched automatically from Semantic Scholar [3].

3. Dolos. Dolos is an open-source source code plagiarism detection tool developed at Ghent University for programming exercises. It is a web app that analyses source code files for similarities between them and presents interactive visualisations and dashboards. Like Dolos, our project shows its analysis on a web dashboard, but it checks the text of PDF documents rather than source code [4].

4. Winnowing: Local Algorithms for Document Fingerprinting. Schleimer, Wilkerson and Aiken introduced winnowing, an efficient local fingerprinting algorithm for identifying copying, including small partial copies, within large sets of documents; the same work reports experience with MOSS. A fingerprinting method selects a subset of the hashes of overlapping k-grams. Our project also works with overlapping n-grams, but of words, and keeps every trigram with its positions in a hash table [2].

5. Turnitin. Turnitin is a commercial similarity-checking service used by educational institutions to compare submitted academic work with a collection of existing sources and to produce a similarity report. Our project follows the same idea of reporting a similarity percentage and matched passages, but it works on a single uploaded PDF and uses reference papers found through Semantic Scholar [5].

6. Copyleaks. Copyleaks is a commercial, web-based plagiarism detection service. Our project is a smaller academic implementation whose purpose is to demonstrate the Data Structures and Algorithms (hash table, KMP and Rabin-Karp) behind such a check [6].

7. Knuth-Morris-Pratt (KMP) string matching. Knuth, Morris and Pratt described a pattern matching algorithm that does not move backwards in the text after a mismatch. It pre-computes a longest-prefix-suffix (LPS) table for the pattern so that, after a mismatch, the pattern can be shifted without moving backwards in the text. This algorithm is implemented in our project and used for the algorithm comparison [7].

8. Rabin-Karp string matching. Karp and Rabin proposed a pattern matching algorithm that compares hash values of the pattern and of each window of the text, and checks the characters only when the hash values are equal. The hash of the next window is obtained from the previous one by a rolling update. This algorithm is also implemented in our project and compared with KMP [8].

4. Problem Statement and Objectives

Problem Statement: To build a word-level plagiarism checker that compares an uploaded PDF document with automatically retrieved reference papers and reports the matching text and similarity percentage using data structures and algorithms.

Problem Description: Manually comparing a document with existing research papers is time-consuming. This project automates the comparison by extracting text from PDFs, matching word trigrams through a hash table and showing the matched passages and similarity percentage on a dashboard.

Objectives:

Identify the Project's Purpose: The project was initiated to automate the word-level comparison of an uploaded PDF document with reference papers and to apply core data structures and algorithms (hash table, KMP and Rabin-Karp) to this task. The objectives of the project are:

- To extract the text of an uploaded PDF document and tokenise it into lowercase words.
- To generate word-level n-grams (trigrams, $n = 3$) from the uploaded document and from the reference documents.
- To implement a hash table with separate chaining that stores the trigrams of a reference document together with their positions.
- To find reference papers automatically through the Semantic Scholar API, using a search query built from the most frequent words of the uploaded document, and to download their open-access PDFs.
- To calculate the number of matched words and the similarity percentage, and to list the matched text with its position in the uploaded document and in the reference.
- To implement the KMP and Rabin-Karp algorithms and compare their matches found, comparisons and execution time on the same trigrams.
- To provide a Java backend and a React dashboard (PlagiaSense) that uploads the PDF and displays the plagiarism score, matched text, DSA algorithm comparison and reference papers.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Desktop / laptop computer running Windows (the project was developed and run on a Windows system).
- Internet connection (required for searching Semantic Scholar and downloading the reference paper PDFs).

Software Requirements:

- **Operating System:** Windows.
- **Programming languages:** Java for the backend (project class files are compiled for Java 25) and JavaScript (JSX) for the frontend.
- **Java libraries:** Apache PDFBox 3.0.8 (with FontBox 3.0.8, PDFBox-IO 3.0.8 and Commons Logging 1.4.0) for PDF text extraction and Gson 2.13.2 for JSON; the built-in com.sun.net.httpserver package is used for the HTTP server.
- **Frontend framework and tools:** React 18.3.1, Vite 5 and @vitejs/plugin-react 4.3.4, run with Node.js and npm.
- **External service:** Semantic Scholar Graph API (accessed with an API key supplied in the environment variable SEMANTIC_SCHOLAR_API_KEY).
- **Development environment:** Visual Studio Code.
- **Web browser:** To open the PlagiaSense dashboard (<http://localhost:5173>), which communicates with the backend at <http://localhost:8080>.

6. IMPLEMENTATION

The project is implemented as a Java backend and a React frontend. The backend source is organised into the packages listed below.

Package	Classes	Purpose
dsa	HashTable, KMP, RabinKarp, AlgorithmStats	Hash table with separate chaining; KMP and Rabin-Karp search with statistics
processing	TextProcessor, NGramGenerator, QueryBuilder, PDFTextExtractor, PDFDownloader	Tokenising, n-gram generation, search query building, PDF text extraction and PDF download
search	SemanticScholarClient	Searches Semantic Scholar for reference papers
plagiarism	DocumentIndexer, PlagiarismEngine, MatchDetector, MultiSourceAnalyzer	Indexing of the reference document, matching, similarity calculation and DSA comparison
model	MatchResult, PlagiarismResult, DSAComparisonResult, ReferencePaperAnalysisResult, DashboardAnalysisResponse	Result objects returned to the dashboard as JSON
server	DashboardServer	HTTP server with the /api/health and /api/analyze endpoints

Working of the system

1. The user uploads a PDF in the PlagiaSense dashboard; the dashboard sends it to the backend (POST /api/analyze) and the text of the PDF is extracted with Apache PDFBox.
2. QueryBuilder builds a search query from the five most frequent words of the document that are longer than three characters and are not in its small stop-word list.
3. SemanticScholarClient searches Semantic Scholar (20 results requested, one retry on HTTP 429). For each result that has an open-access PDF, the PDF is downloaded and its text is extracted. Up to three reference papers are collected within a 45-second search budget.
4. For every reference paper a PlagiarismEngine (hash table size 100, $n = 3$) indexes the reference text and analyses the uploaded text, giving the matched words, matched text and similarity percentage.
5. KMP and Rabin-Karp are run on the same trigrams against the reference text and their statistics are recorded.
6. The reference paper with the highest similarity gives the overall score, matched text and DSA comparison; the response, including every reference paper analysed, is returned as

JSON and shown on the dashboard. If no reference paper can be obtained, a locally stored source.pdf is used as a fallback.

Main modules and code

The code of the four main modules is given below (comments are trimmed and some long statements are written on fewer lines; the logic is as in the project).

Module 1: Text Processing, N-gram Generation and Query Building

TextProcessor converts the text to lowercase, removes every character other than letters, digits and spaces, and splits it into words. NGramGenerator joins n consecutive words into overlapping n-grams. QueryBuilder counts the words longer than three characters that are not stop words and joins the five most frequent ones into the search query.

```
// TextProcessor.java
public static List<String> tokenize(String text) {
    text = text.toLowerCase();
    text = text.replaceAll("[^a-z0-9\\s]", "");
    String[] words = text.split("\\s+");
    List<String> tokens = new ArrayList<>();
    for (String word : words) {
        if (!word.isEmpty()) {
            tokens.add(word);
        }
    }
    return tokens;
}

// NGramGenerator.java
public static List<String> generateNGrams(List<String> words, int n) {
    List<String> ngrams = new ArrayList<>();
    for (int i = 0; i <= words.size() - n; i++) {
        StringBuilder ngram = new StringBuilder();
        for (int j = 0; j < n; j++) {
            ngram.append(words.get(i + j));
            if (j < n - 1) {
                ngram.append(" ");
            }
        }
        ngrams.add(ngram.toString());
    }
    return ngrams;
}

// QueryBuilder.java
public static String buildQuery(String documentText) {
    if (documentText == null || documentText.isBlank()) {
        return "";
    }
    List<String> words = TextProcessor.tokenize(documentText);
    Map<String, Integer> counts = new HashMap<>();
    for (String word : words) {
        if (word.length() > 3 && !STOPWORDS.contains(word)) {
            counts.merge(word, 1, Integer::sum);
        }
    }
    return counts.entrySet().stream()
        .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
        .limit(MAX_TERMS) // MAX_TERMS = 5
        .map(Map.Entry::getKey)
        .reduce((a, b) -> a + " " + b)
```

```

        .orElse("");
    }
}

```

Module 2: Hash Table Indexing

HashTable is a hash table written from scratch. The index is computed as $\text{hash} = (\text{hash} \times 31 + \text{character}) \bmod \text{size}$, and collisions are handled by separate chaining (a linked list of nodes in each bucket). Each node stores a trigram and the list of positions at which it occurs. DocumentIndexer tokenises the reference text, generates its trigrams and inserts every trigram with its position.

```

// HashTable.java
public class HashTable {
    private static class Node {
        String key;
        List<Integer> positions;
        Node next;
        Node(String key, int position) {
            this.key = key;
            positions = new ArrayList<>();
            positions.add(position);
            next = null;
        }
    }
    private Node[] table;
    private int size;

    public HashTable(int size) {
        this.size = size;
        table = new Node[size];
    }

    private int hash(String key) {
        int hash = 0;
        for (int i = 0; i < key.length(); i++) {
            hash = (hash * 31 + key.charAt(i)) % size;
        }
        return hash;
    }

    public void insert(String key, int position) {
        int index = hash(key);
        Node current = table[index];
        if (current == null) { // bucket is empty
            table[index] = new Node(key, position);
            return;
        }
        while (true) { // search for existing key
            if (current.key.equals(key)) {
                current.positions.add(position);
                return;
            }
            if (current.next == null) {
                break;
            }
            current = current.next;
        }
        current.next = new Node(key, position); // collision: add new node
    }

    public List<Integer> search(String key) {
        int index = hash(key);
        Node current = table[index];
        while (current != null) {
            if (current.key.equals(key)) {

```

```

        return current.positions;
    }
    current = current.next;
}
return new ArrayList<>();
}
}

// DocumentIndexer.java
public void indexDocument(String text, int n) {
    List<String> words = TextProcessor.tokenize(text);
    List<String> ngrams = NGramGenerator.generateNGrams(words, n);
    for (int i = 0; i < ngrams.size(); i++) {
        hashTable.insert(ngrams.get(i), i);
    }
}
}

```

Module 3: Plagiarism Detection and Similarity Calculation

PlagiarismEngine.analyze() generates the trigrams of the uploaded document and searches each of them in the hash table of the reference document. For every match, the three words of the trigram are marked as matched and a MatchResult (matched text, position in the uploaded document, position in the reference) is stored. The similarity is the percentage of matched words out of the total words of the uploaded document. The DSA comparison is then run on the same trigrams.

```

// PlagiarismEngine.java
public PlagiarismResult analyze(String userText) {
    List<String> userWords = TextProcessor.tokenize(userText);
    List<String> userNGrams = NGramGenerator.generateNGrams(userWords, n);
    boolean[] matchedPositions = new boolean[userWords.size()];
    List<MatchResult> matches = new ArrayList<>();

    for (int i = 0; i < userNGrams.size(); i++) {
        String ngram = userNGrams.get(i);
        List<Integer> sourcePositions = indexer.search(ngram);
        if (!sourcePositions.isEmpty()) {
            int sourcePosition = sourcePositions.get(0);
            for (int j = 0; j < n; j++) {
                if (i + j < matchedPositions.length) {
                    matchedPositions[i + j] = true;
                }
            }
            matches.add(new MatchResult(ngram, i, sourcePosition, 100.0));
        }
    }

    int matchedWords = 0;
    for (boolean matched : matchedPositions) {
        if (matched) {
            matchedWords++;
        }
    }
    int totalWords = userWords.size();
    double similarity = 0.0;
    if (totalWords > 0) {
        similarity = ((double) matchedWords / totalWords) * 100;
    }

    dsaResult = runDSAComparison(userNGrams); // KMP vs Rabin-Karp
    return new PlagiarismResult(totalWords, matchedWords, similarity, matches);
}

```

Module 4: KMP and Rabin-Karp Algorithm Comparison

Both algorithms search every trigram of the uploaded document in the full text of the reference paper and return an AlgorithmStats object (match position, character comparisons, hash comparisons, execution time). KMP uses the LPS (longest prefix-suffix) array to avoid re-comparing characters. Rabin-Karp uses $BASE = 256$ and $PRIME = 101$, compares rolling hash values first and compares characters only when the hashes are equal. runDSAComparison() adds up the statistics of all trigrams and records which algorithm took less total time.

```
// KMP.java (null / empty / pattern-longer-than-text checks omitted)
public static AlgorithmStats searchWithStats(String text, String pattern) {
    long startTime = System.nanoTime();
    int[] lps = buildLPS(pattern);
    int i = 0;
    int j = 0;
    long comparisons = 0;
    while (i < text.length()) {
        comparisons++;
        if (text.charAt(i) == pattern.charAt(j)) {
            i++;
            j++;
            if (j == pattern.length()) {
                return new AlgorithmStats(i - j, comparisons, 0,
                    System.nanoTime() - startTime);
            }
        } else {
            if (j != 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }
    return new AlgorithmStats(-1, comparisons, 0, System.nanoTime() - startTime);
}

private static int[] buildLPS(String pattern) {
    int[] lps = new int[pattern.length()];
    int length = 0;
    int i = 1;
    while (i < pattern.length()) {
        if (pattern.charAt(i) == pattern.charAt(length)) {
            length++;
            lps[i] = length;
            i++;
        } else {
            if (length != 0) {
                length = lps[length - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
    return lps;
}
```

```
// RabinKarp.java (null / empty / pattern-longer-than-text checks omitted)
private static final int BASE = 256;
private static final int PRIME = 101;
public static AlgorithmStats searchWithStats(String text, String pattern) {
    long startTime = System.nanoTime();
    int patternLength = pattern.length();
    int textLength = text.length();
    long patternHash = 0;
```

```

    long textHash = 0;
    long highestPower = 1;
    long hashComparisons = 0;
    long characterComparisons = 0;
    for (int i = 0; i < patternLength - 1; i++) {
        highestPower = (highestPower * BASE) % PRIME;
    }
    for (int i = 0; i < patternLength; i++) {
        patternHash = (BASE * patternHash + pattern.charAt(i)) % PRIME;
        textHash = (BASE * textHash + text.charAt(i)) % PRIME;
    }
    for (int i = 0; i <= textLength - patternLength; i++) {
        hashComparisons++;
        if (patternHash == textHash) {
            boolean match = true;
            for (int j = 0; j < patternLength; j++) {
                characterComparisons++;
                if (text.charAt(i + j) != pattern.charAt(j)) {
                    match = false;
                    break;
                }
            }
            if (match) {
                return new AlgorithmStats(i, characterComparisons,
                    hashComparisons, System.nanoTime() - startTime);
            }
        }
        if (i < textLength - patternLength) { // roll the hash
            textHash = (BASE * (textHash - text.charAt(i) * highestPower)
                + text.charAt(i + patternLength)) % PRIME;
            if (textHash < 0) {
                textHash += PRIME;
            }
        }
    }
    return new AlgorithmStats(-1, characterComparisons, hashComparisons,
        System.nanoTime() - startTime);
}

```

```

// PlagiarismEngine.runDSAComparison() (loop over every trigram of the user text)
for (String ngram : userNGrams) {
    AlgorithmStats kmpStats = KMP.searchWithStats(sourceText, ngram);
    totalKMPComparisons += kmpStats.getComparisons();
    totalKMPTime += kmpStats.getExecutionTime();
    if (kmpStats.getMatchPosition() != -1) {
        kmpMatches++;
    }

    AlgorithmStats rabinStats = RabinKarp.searchWithStats(sourceText, ngram);
    totalRabinCharacterComparisons += rabinStats.getComparisons();
    totalRabinHashComparisons += rabinStats.getHashComparisons();
    totalRabinTime += rabinStats.getExecutionTime();
    if (rabinStats.getMatchPosition() != -1) {
        rabinMatches++;
    }
}
double kmpTimeMs = totalKMPTime / 1_000_000.0;
double rabinTimeMs = totalRabinTime / 1_000_000.0;
String observation;
if (kmpTimeMs < rabinTimeMs) {
    observation = "KMP was faster for this test case.";
} else if (rabinTimeMs < kmpTimeMs) {
    observation = "Rabin-Karp was faster for this test case.";
} else {
    observation = "Both algorithms had similar execution time.";
}

```

7. RESULTS & DISCUSSION

The application was run with the Java backend (port 8080) and the PlagiaSense dashboard (port 5173). The screens of the dashboard and their explanation are given below.

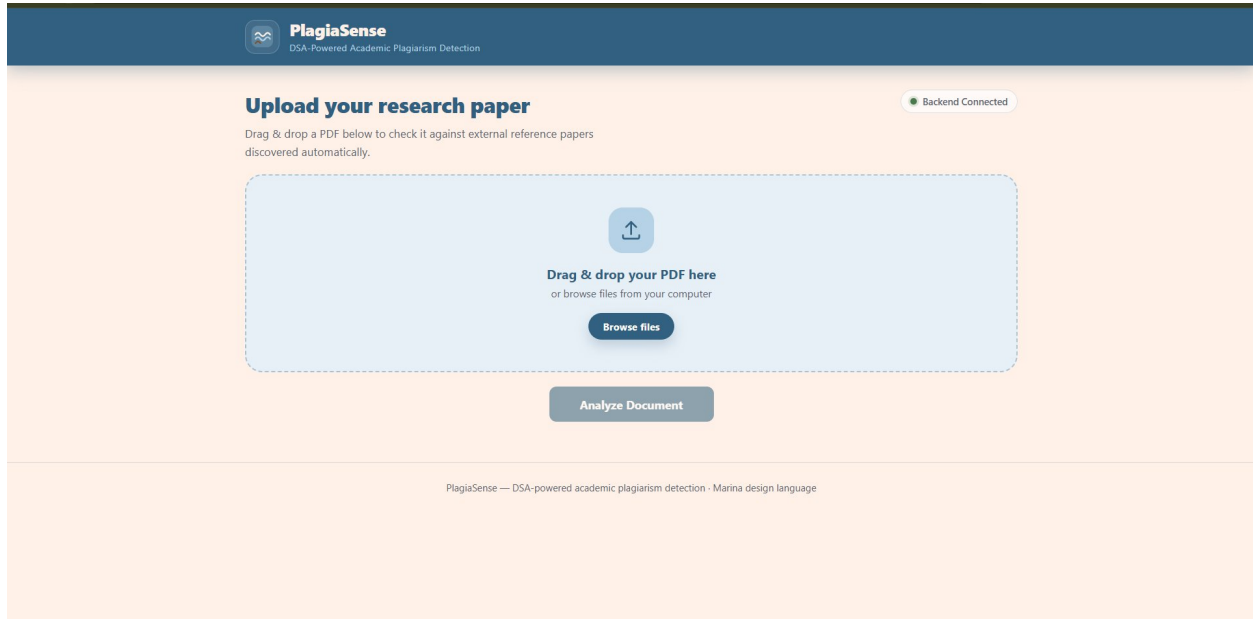


Figure 7.1: PlagiaSense home page with the PDF upload area

The dashboard shows the “Upload your research paper” section with a drag-and-drop upload zone that accepts only PDF files, and the “Analyze Document” button. The status pill at the top right shows “Backend Connected” when the backend health check (/api/health) succeeds.

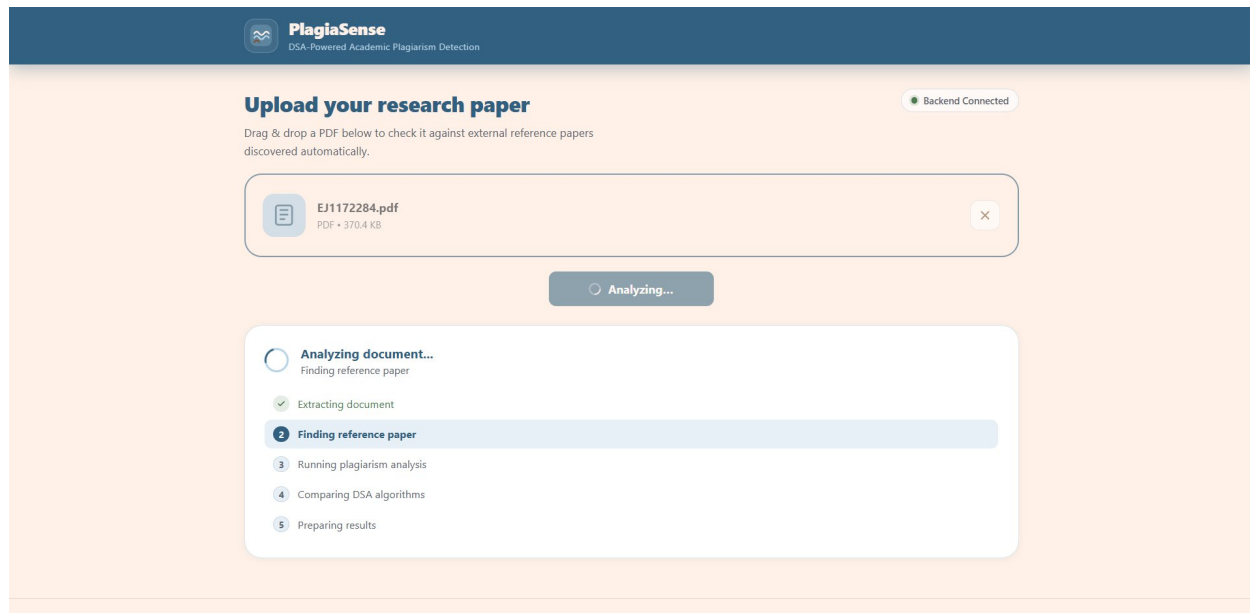


Figure 7.2: Analysis in progress

After “Analyze Document” is clicked, the dashboard shows “Analyzing document...” with five stages: Extracting document, Finding reference paper, Running plagiarism analysis, Comparing DSA algorithms and Preparing results. The current stage is highlighted while the backend extracts the text, finds reference papers and runs the analysis.

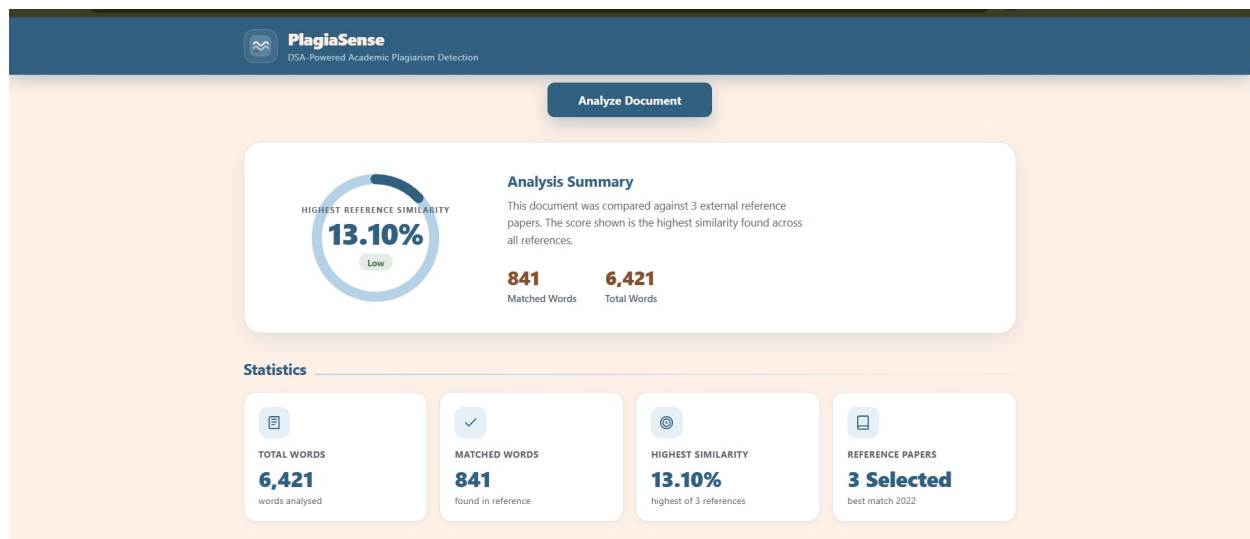


Figure 7.3: Analysis summary and statistics

The ring shows the plagiarism score (highest reference similarity) with a Low / Moderate / High label (below 15%, below 40%, otherwise High), together with the matched words and total words. The Statistics cards show Total Words, Matched Words, Highest Similarity and the number of Reference Papers selected.


PlagiaSense
DSA-Powered Academic Plagiarism Detection

Matched Text
383 matching passages found

Matched Text — Highest Similarity Reference

Showing 12 of 383

MATCHED TEXT	USER POSITION	SOURCE POSITION	MATCH %
"use of mobile"	17	1,250	100.00%
"of mobile devices"	18	315	100.00%
"mobile devices for"	19	7	100.00%
"devices for english"	20	8	100.00%
"for english language"	21	9	100.00%
"the results of"	42	1,203	100.00%
"results of a"	43	1,204	100.00%
"of a study"	44	1,385	100.00%

Show more

Figure 7.4: Matched text table

The Matched Text table lists each matching trigram with its position in the uploaded document, its position in the reference paper and the match percentage. The table can be filtered by text or position and shows 12 rows at a time with a “Show more” button.

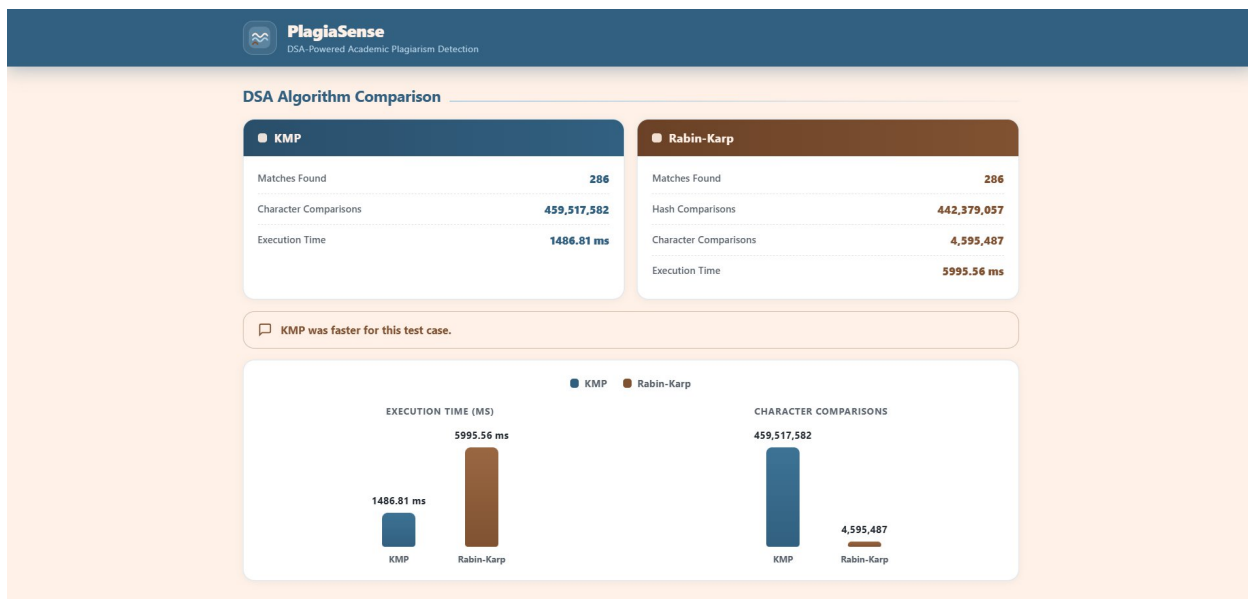


Figure 7.5: DSA algorithm comparison (KMP vs Rabin-Karp)

The comparison shows, for KMP and for Rabin-Karp, the matches found, character comparisons (and hash comparisons for Rabin-Karp) and execution time, followed by the observation on which algorithm was faster. Two bar charts compare the execution time and the character comparisons of the two algorithms.


PlagiaSense
DSA-Powered Academic Plagiarism Detection

Reference Papers
3 external references analyzed

1

Factors Influencing Vietnamese Teenagers' Intention to Use Mobile Devices for English Language Learning
 Q. Hoàng, T. Pham, Quang Dang, Thao P. Nguyen
Published 2022

16ae5543b82e4a47baea9e3ffff9ad155de37...

SIMILARITY

13.10%

MATCHED WORDS

841

KMP MATCHES

286

RABIN-KARP MATCHES

286

View on Semantic Scholar
Open PDF

2

Mobile-assisted language learning in Vietnam: Insights from undergraduate English majors
 Thanh Thi Vu
Published 2025

ae491b8a7d7ae5548cc2a8536a79524ef9310b...

SIMILARITY

9.84%

MATCHED WORDS

632

KMP MATCHES

226

RABIN-KARP MATCHES

226

View on Semantic Scholar
Open PDF

3

MOBILE-ASSISTED LANGUAGE LEARNING IN BUSINESS ENGLISH CONTEXTS
 K. Hnatyk, M. Fabian
Published 2025

f038cd3ff76f25f46e2a7d2f25d9452792f2b...

SIMILARITY

2.32%

MATCHED WORDS

149

KMP MATCHES

40

RABIN-KARP MATCHES

40

View on Semantic Scholar
Open PDF

Figure 7.6: Reference papers analysed

The Reference Papers section lists each reference paper found through Semantic Scholar with its title, authors, year, similarity, matched words and the KMP and Rabin-Karp matches. Each paper has the buttons “View on Semantic Scholar” and “Open PDF”.

8. CONCLUSION & FUTURE SCOPE

Conclusion

The objectives of the project were met. A Java backend and a React dashboard were developed in which an uploaded PDF is converted into word tokens and word-level trigrams, and its text is compared with reference papers found automatically through Semantic Scholar. A hash table with separate chaining, written from scratch, stores the trigrams of a reference paper and gives the matched words, the matched text with positions and the similarity percentage. The KMP and Rabin-Karp algorithms were implemented and their matches, comparisons and execution time were compared on the same data, and all of these results are displayed on the PlagiaSense dashboard.

Future Scope

- **Near term:** remove the temporary source.pdf fallback so that the analysis always uses the reference papers obtained from Semantic Scholar.
- **Medium term:** make the n-gram size (currently 3), the hash table size (currently 100) and the number of reference papers (currently up to 3) configurable instead of fixed in the code.
- **Long term:** accept document formats other than PDF for the uploaded document.

9. REFERENCES

1. Stanford University, “Moss: A System for Detecting Software Similarity,” [Online]. Available: <https://theory.stanford.edu/~aiken/moss/>
2. S. Schleimer, D. S. Wilkerson and A. Aiken, “Winnowing: Local Algorithms for Document Fingerprinting,” in Proc. ACM SIGMOD International Conference on Management of Data, 2003. <https://doi.org/10.1145/872757.872770>
3. JPlag, “JPlag – Detecting Source Code Plagiarism,” GitHub repository. [Online]. Available: <https://github.com/jplag/JPlag>
4. Dodona Team, Ghent University, “Dolos – Source code plagiarism detection,” [Online]. Available: <https://dolos.ugent.be> and <https://github.com/dodona-edu/dolos>
5. Turnitin, “Turnitin – Similarity checking for academic integrity,” [Online]. Available: <https://www.turnitin.com>
6. Copyleaks, “Copyleaks – Plagiarism detection,” [Online]. Available: <https://copyleaks.com>
7. D. E. Knuth, J. H. Morris and V. R. Pratt, “Fast Pattern Matching in Strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977. <https://doi.org/10.1137/0206024>
8. R. M. Karp and M. O. Rabin, “Efficient randomized pattern-matching algorithms,” IBM Journal of Research and Development, vol. 31, no. 2, pp. 249–260, 1987. <https://doi.org/10.1147/rd.312.0249>
9. Allen Institute for AI, “Semantic Scholar Academic Graph API,” [Online]. Available: <https://api.semanticscholar.org/api-docs/>
10. The Apache Software Foundation, “Apache PDFBox – A Java PDF Library,” [Online]. Available: <https://pdfbox.apache.org/>
11. Google, “Gson – A Java serialization/deserialization library to convert Java Objects into JSON and back,” [Online]. Available: <https://github.com/google/gson>
12. Meta Platforms, Inc., “React – The library for web and native user interfaces,” [Online]. Available: <https://react.dev>; and Vite, “Vite – Next Generation Frontend Tooling,” [Online]. Available: <https://vite.dev>